Provisioning Source Libraries for Developers

TEKsystems Global Services

WORKFORCE DEVELOPMENT

TYPICAL DATA SETS NEEDED BY DEVELOPERS

Depends on company standards and team organization. This is a generic example.

Libraries	Dev	Test	Prod
Source Code	Individual	Team	None
Includes/Copybooks	Individual	Team	None
Code Snippets	Individual	Team(?)	None
SQL Scripts	Individual	Team	No dev access
REXX	Individual	Team/App	No dev access
JCL	Individual	Team/App	No dev access
Executables	Individual	Team/App	No dev access

CONSIDERATIONS FOR DATA SET CREATION

Depends on company standards and team organization. This is a generic example.

Libraries	Dev	Test	Prod
Sour			e
Inclu			e
Code			e
SQL			dev ess
REX			dev ess
JCL			dev access
Executables	Individual	Team/App	No dev access

Which data sets are created and how they are named depend on:

- How best to enable developers and testers to do their work smoothly
- Managing system security
- Regulatory compliance

CONSIDERATIONS FOR DATA SET CREATION

Depends on company standards and team organization. This is a generic example.

Scripts, executables, and jobstreams must exist in the production environment, but developers and testers must not have access to them.

You control it through a combination of naming conventions for data sets and RACF definitions for groups and users to access resources that have specific qualifiers.

Similarly, people and jobs that have access to production data must not have access to development tools.

This is a regulatory requirement.

Prod
None
None
None
No dev access
No dev access
No dev access
No dev access

CONSIDERATIONS FOR DATA SET CREATION

Depends on company standards and team organization. This is a generic example.

Libraries	Dev	Test	Prod
Source Code	Individual	Team	None
REXX	Individual	Team/App	None
JCL	Individual	Team/App	None
Executables	Individual	Team/App	No dev access
			No dev access
			No dev access
			No dev access

Source artifacts that could be used to modify production applications cannot exist in the production environment.

CONSIDERATIONS FOR DATA SET CREATION

Depends on company standards and team organization. This is a generic example.

Libraries	Dev	Test	Prod
Source Code	Individual	Team	None
Includes/Copybooks	Individual	Team	None
Code Snippets	Individual	Team(?)	None
SQL Sc	dual	Team	No dev access
REXX	dual	Team/App	No dev access
JCL	dual	Team/App	No dev access
Executables	Individual	Team/App	No dev access

It's often necessary to have multiples of the same type of library with separate data sets by individual, team, and/or application.

IMPORTANCE OF NAMING CONVENTIONS

A lot of the activity on z/OS systems is driven by the names of the data sets involved in that activity. Having consistent naming conventions for data sets helps with:

- Organizing IT assets by application, environment, business function, or team
- Managing access rights
- Satisfying regulatory requirements such as Sarbanes-Oxley
- Automating routine operational procedures, deployments, testing
- Simplifying application migration, upgrades, backup/restore
- Simplifying observability, log aggregation, and system monitoring
- Tracking change history
- Managing disaster recovery
- Identifying connections/dependencies between systems
- Detecting unusual or unexpected system activity
- Onboarding new employees and removing userids no longer in use

NO INDUSTRY-STANDARD NAMING CONVENTIONS

"The Tao that can be told is not the eternal Tao; the name that can be named is not the eternal name."
- Laozi, *Tao Te Ching*

Every company has its own naming conventions for z/OS data sets and its own rationale for mapping users and groups to named assets.

When you complete basic training, you will learn the naming conventions used in your company and the rationale for them.

When you are provisioning environments, installing software, or otherwise working with the z/OS systems, the naming conventions will guide much of your work.

For training purposes, we will define naming conventions to use in the lab exercises. Bear in mind these conventions are arbitrary.

NAMING CONVENTIONS FOR LABS

Libraries	Dev	Test	Prod
Source Code	Individual	N/A	None
Includes/Copybooks	Individual	N/A	None
Code Snippets	Individual	N/A	None
SQL Scripts	Individual	N/A	No dev access
REXX	Individual	N/A	No dev access
JCL	Individual	N/A	No dev access
Executables	Individual	N/A	No dev access

NAMING CONVENTIONS FOR LABS

Libraries	HLQ	MLQ	Name
Source Code	<userid>	DEV or TST	<lang>
Includes/Copybooks	<userid>	DEV or TST	<lang-dep>
Code Snippets	<userid>	SNIPPETS	<lang>
SQL Scripts	<userid>	DEV or TST	SQL
REXX	<userid>	DEV or TST	REXX
JCL	<userid>	DEV or TST	JCL
JCL Procedures	<userid>	DEV or TST	PROCLIB
Executables	<userid>	DEV or TST	PROGLIB

Legend
<userid> = your userid
<lang> = ASM, COBOL, PLI
<lang-dep> ASM = MACLIB, COBOL = COPYLIB, PLI = INCLUDE

SAMPLE DATA SET NAMES

Libraries for user ABCDEF

```
ABCDEF.DEV.COBOL          ABCDEF.DEV.ASM          ABCDEF.DEV.PLI
ABCDEF.DEV.COPYLIB        ABCDEF.DEV.MACLIB       ABCDEF.DEV.INCLUDE
ABCDEF.TST.COBOL          ABCDEF.TST.ASM          ABCDEF.TST.PLI
ABCDEF.TST.COPYLIB        ABCDEF.TST.MACLIB       ABCDEF.TST.INCLUDE
ABCDEF.SNIPPETS.COBOL     ABCDEF.SNIPPETS.ASM     ABCDEF.SNIPPETS.PLI
ABCDEF.DEV.SQL
ABCDEF.TST.SQL
ABCDEF.DEV.REXX           ABCDEF.SNIPPETS.REXX
ABCDEF.TST.REXX
ABCDEF.DEV.JCL            ABCDEF.SNIPPETS.JCL
ABCDEF.TST.JCL
ABCDEF.DEV.PROCLIB
ABCDEF.TST.PROCLIB
ABCDEF.DEV.PROGLIB
ABCDEF.TST.PROGLIB
```

NEW THINGS TO LEARN

You know how to write JCL that safely deletes a sequential dataset and then allocates a new sequential dataset with the same name.

A job to allocate a Library or PDSe has the same general shape.

What's new:

- Coding a DD statement that specifies the attributes of a Library rather than a sequential data set
- Factoring out the repetitive JCL statements into a re-usable procedure, called a Catalogued Procedure or PROC.
- Structuring the job(s) to use the PROC instead of repetitively specifying nearly-identical JCL statements.

ALLOCATING A LIBRARY (PDSE) WITH JCL

```
EDIT       IBMUSER.LAB.JCL(CRSRCLIB) - 01.00            Columns 00001 00072
Command ===>                                             Scroll ===> CSR
****** ***************************** Top of Data ******************************
000001 //IBMUSERS JOB ,'ALLOC SOURCE LIBRARY',
000002 //            CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
000003 //            NOTIFY=&SYSUID
000004 //CLEANUP  EXEC PGM=IEFBR14
000005 //DD1      DD DSN=&SYSUID..DEV.JCL,
000006 //            DISP=(MOD,DELETE,DELETE),
000007 //            UNIT=SYSDA,VOL=SER=USRVS1,
000008 //            SPACE=(TRK,(1,0)),
000009 //            DCB=(DSORG=PO,RECFM=FB,LRECL=80,BLKSIZE=27920),
000010 //            DSNTYPE=LIBRARY
000011 //ALLOC    EXEC PGM=IEFBR14
000012 //DD1      DD DSN=&SYSUID..DEV.JCL,
000013 //            DISP=(NEW,CATLG,DELETE),
000014 //            UNIT=SYSDA,VOL=SER=USRVS1,
000015 //            SPACE=(TRK,(1,1),RLSE),
000016 //            DCB=(DSORG=PO,RECFM=FB,LRECL=80,BLKSIZE=27920),
000017 //            DSNTYPE=LIBRARY
****** **************************** Bottom of Data ****************************
```

The DD statement parameters to allocate a PDSE or Library are mostly the same as those to allocate a sequential data set.

The differences are in the DSORG subparameter of the DCB parameter, and the addition of the DSNTYPE parameter.

Data Set Organization (DSORG) is Partitioned Organization (PO).

RECFM, LRECL, and BLKSIZE pertain to the members of the Library.

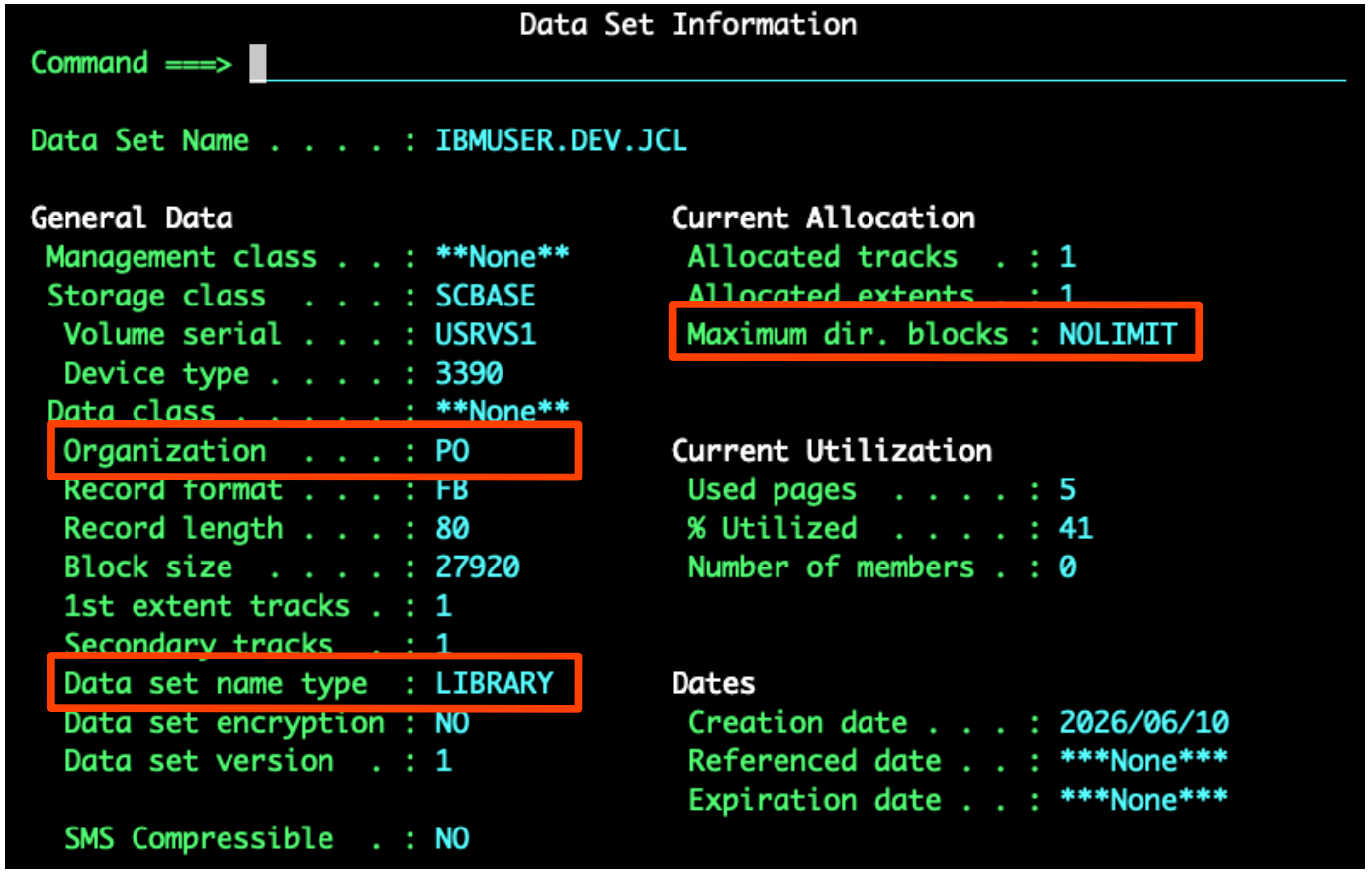
The DSNTYPE=LIBRARY parameter tells the system to allocate a PDSE rather than a PDS. If you omit this, it will allocate a PDS by default.

ALLOCATING A LIBRARY (PDSE) WITH JCL

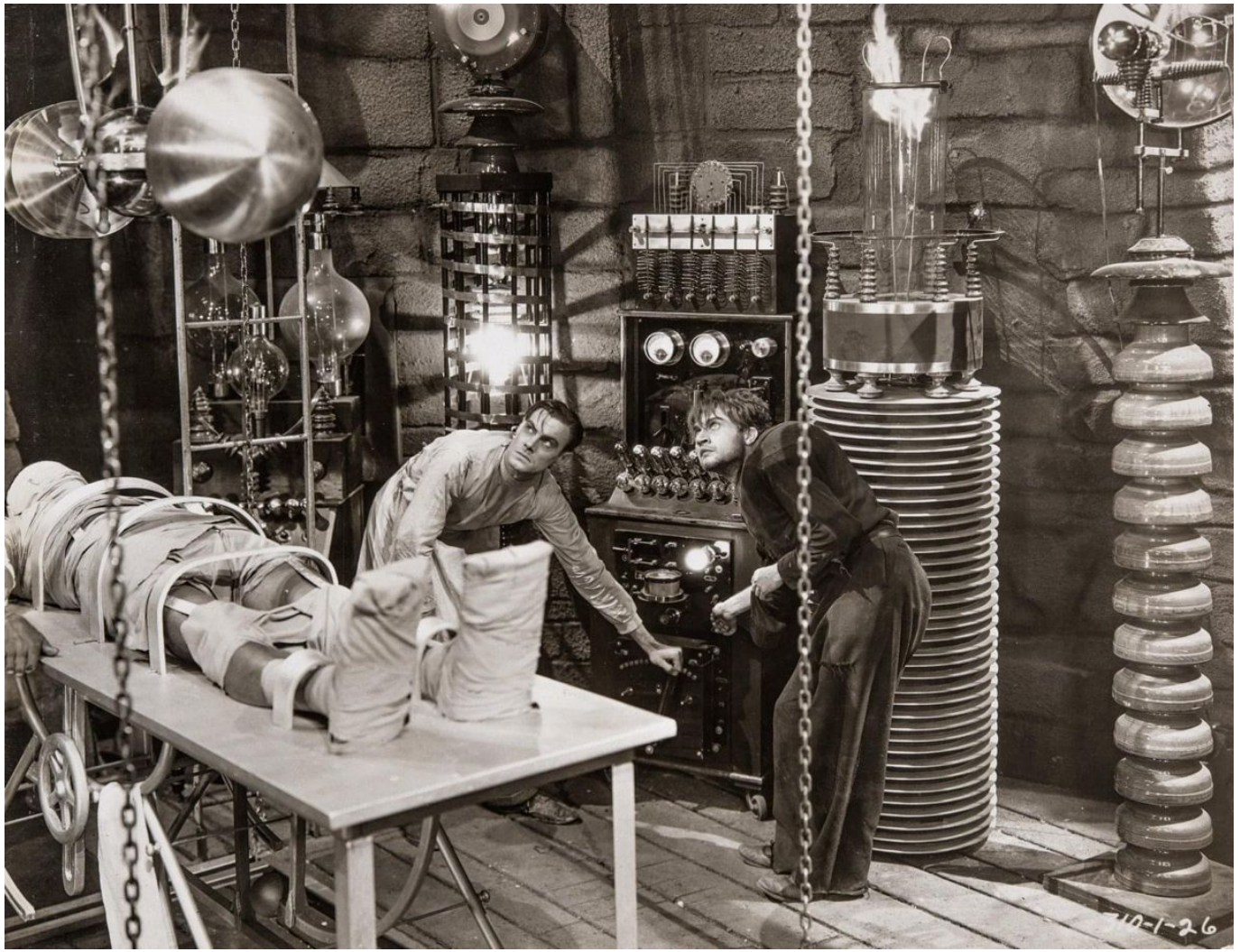
That JCL gets us what we're looking for – a PDSE suitable for "source" members.

On z/OS, all "source" data has to look like 1960s-vintage 80-column punched cards.

Therefore, we can use the same specifications for all the developers' libraries that are used for source of any kind – program source code, REXX scripts, JCL, SQL scripts, configuration files, etc.



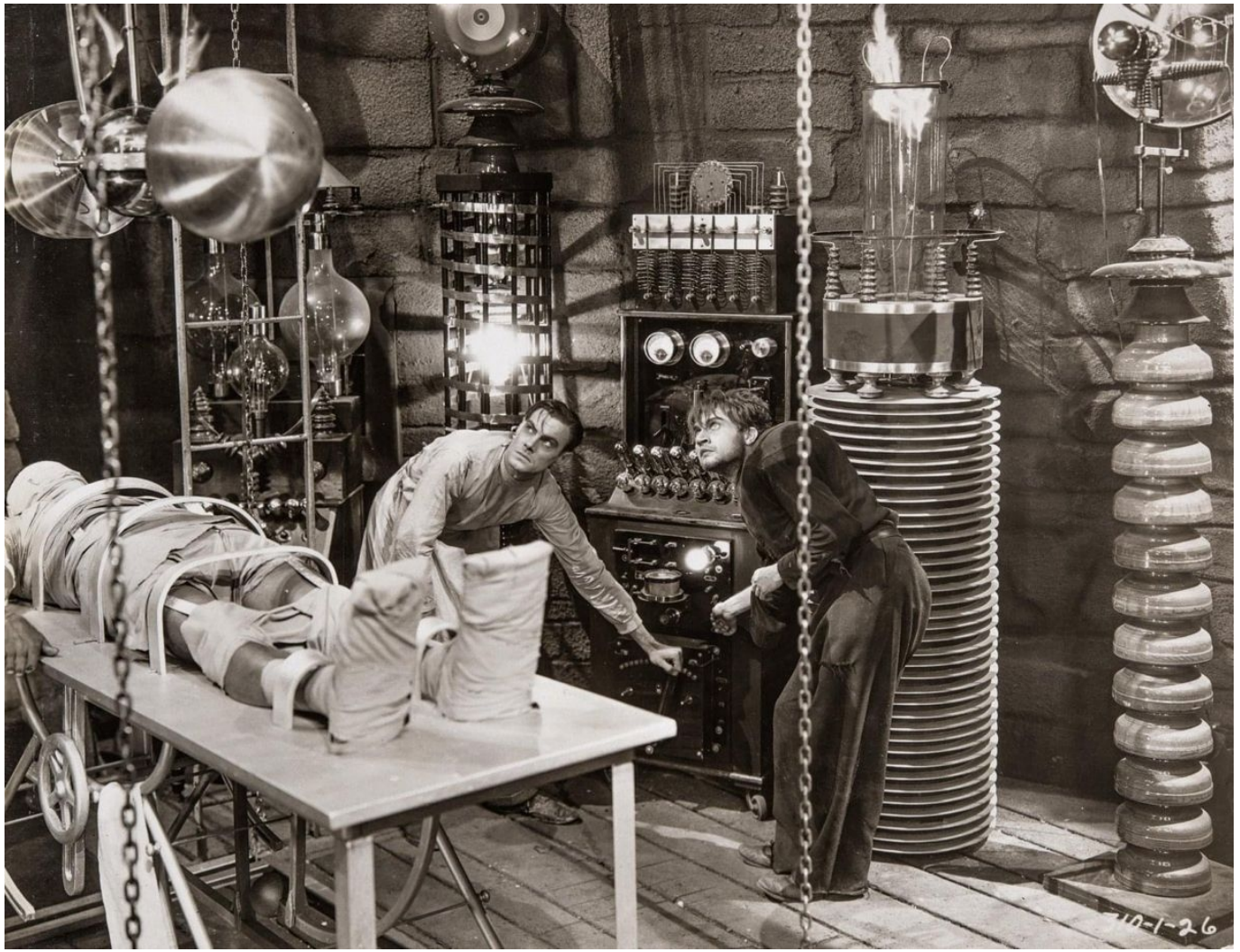
LAB Z/OS 11: CREATE A LIBRARY FOR JCL



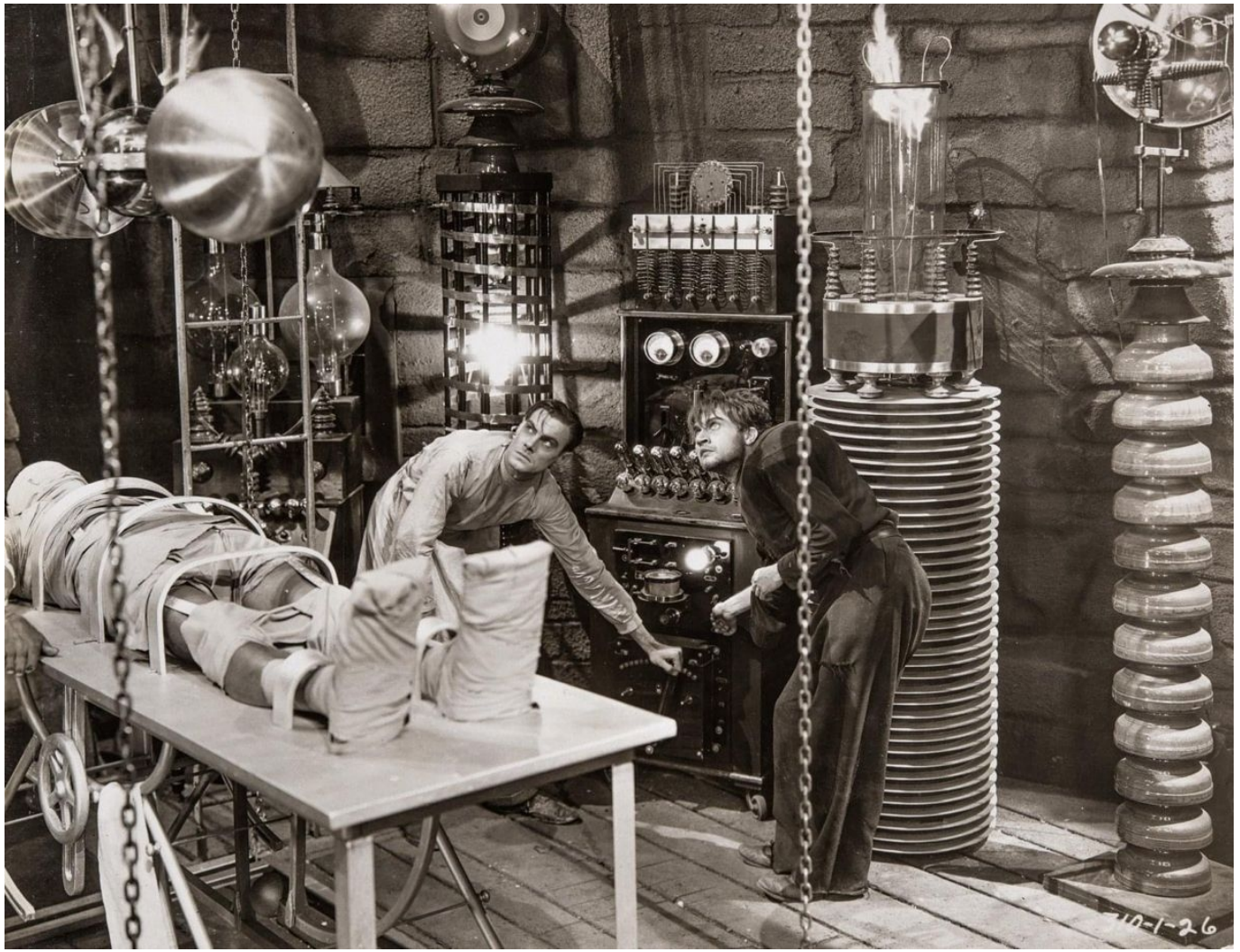
ALLOCATING A MODEL DSCB FOR SOURCE LIBRARIES

Since all source libraries can have the same DCB attributes, we can avoid redundant coding of duplicate DCB parameters in our JCL for allocating libraries. You've done this for sequential data sets, so we'll leave it to you to do the same for PDSE data sets.

LAB Z/OS 12: CREATE A MODEL DSCB FOR LIBRARIES



LAB Z/OS 13: ADD DD'S FOR TST JCL LIBRARY



REFACTORING THE JCL

At this point your JCL to create developer libraries should look something like this (below the JOB statement).

In the CLEANUP step, we coded DD statements for the DEV.JCL library and the TST.JCL library. Everything else about the two DD statements is the same.

Similarly, in the ALLOC step we added a DD statement for TST.JCL.

A programmer would look at this and see duplicate code – an opportunity to refactor. Working in a programming language, they would be inclined to factor out the duplicate bits into a separate routine and then call that routine twice. And we have a lot more than two libraries to allocate.

JCL is not a programming language, but can we do something like that with it?

```
 File  Edit  Edit_Settings  Menu  Utilities  Compilers  Test  Help
EDIT       IBMUSER.LAB.JCL(CRSRCLIB) - 01.02            Columns 00001 00072
Command ===>                                              Scroll ===> CSR
000004 //CLEANUP  EXEC PGM=IEFBR14
000005 //DD1      DD DSN=&SYSUID..DEV.JCL,
000006 //           DISP=(MOD,DELETE,DELETE),
000007 //           UNIT=SYSDA,VOL=SER=USRVS1,
000008 //           SPACE=(TRK,(1,0)),
000009 //           DCB=&SYSUID..MODELDCB.DCBLIB80,
000010 //           DSNTYPE=LIBRARY
000011 //DD2      DD DSN=&SYSUID..TST.JCL,
000012 //           DISP=(MOD,DELETE,DELETE),
000013 //           UNIT=SYSDA,VOL=SER=USRVS1,
000014 //           SPACE=(TRK,(1,0)),
000015 //           DCB=&SYSUID..MODELDCB.DCBLIB80,
000016 //           DSNTYPE=LIBRARY
000017 //ALLOC    EXEC PGM=IEFBR14
000018 //DD1      DD DSN=&SYSUID..DEV.JCL,
000019 //           DISP=(NEW,CATLG,DELETE),
000020 //           UNIT=SYSDA,VOL=SER=USRVS1,
000021 //           SPACE=(TRK,(1,1),RLSE),
000022 //           DCB=&SYSUID..MODELDCB.DCBLIB80,
000023 //           DSNTYPE=LIBRARY
000024 //DD2      DD DSN=&SYSUID..TST.JCL,
000025 //           DISP=(NEW,CATLG,DELETE),
000026 //           UNIT=SYSDA,VOL=SER=USRVS1,
000027 //           SPACE=(TRK,(1,1),RLSE),
000028 //           DCB=&SYSUID..MODELDCB.DCBLIB80,
000029 //           DSNTYPE=LIBRARY
****** ***************************** Bottom of Data ****************************
```

JCL PROCEDURES

It turns out we can. A JCL Procedure, or PROC, is a set of JCL statements that can be invoked on EXEC statements in jobs.

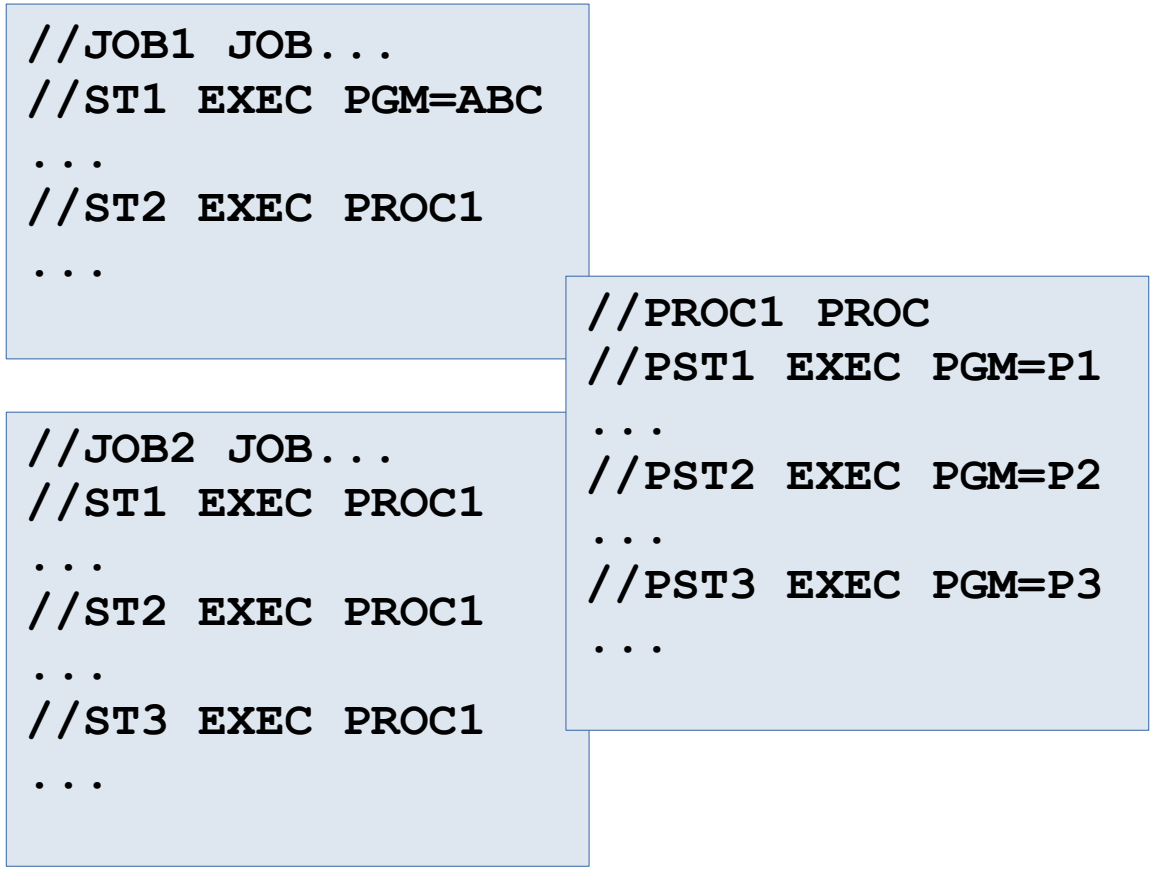
A PROC isn't just a set of statements that are included in the jobstream. The statements can't be arbitrary, like a single DD statement or just a couple of parameters for a DD statement.

A PROC has to specify one or more steps that might be included in a job. If the steps were manually copied into the job, they would work the same.

The EXEC statement can specify any of:

```
EXEC PGM=PROGNAME
EXEC PROC=PROCNAME
EXEC PROCNAME
```

A PROC can be defined inline as part of a jobstream or it can be stored in a library and referenced by multiple jobs, as suggested by the illustration at right.



REFACTORING THE JCL

Looking at these job steps, where do you see duplication that could be factored out into a separate PROC?

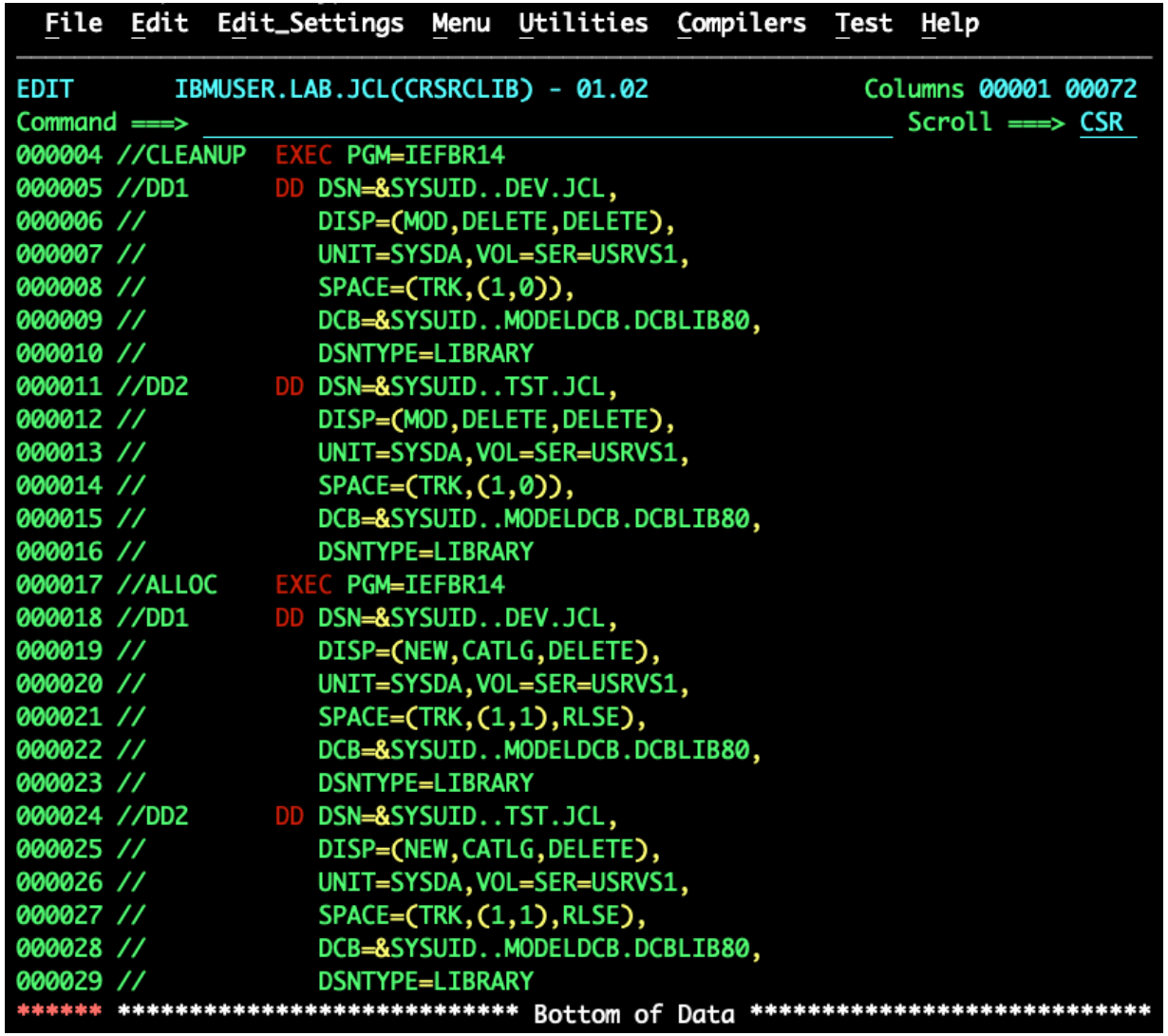
It seems to me that the DD statements in the CLEANUP step are identical except for the data set names.

The DD statements in the ALLOC step are identical except for the data set names, and the SPACE allocation could be different, as well. A developer might need a larger library for COBOL source than for REXX source, for instance.

Otherwise, there's a lot of duplicated code here.

Each unit of work in this job consists of deleting the old library, if it exists, and then allocating a new library with the same name. It's convenient to throw more and more DD statements into each step, but there may be a cleaner way.

Our PROC could consist of a CLEANUP step and an ALLOC step that deal with just one library. Then we could write a job that executes the PROC repeatedly to create several libraries.



REFACTORING THE JCL

Let's begin the refactoring by stripping the JCL down to what we need to delete and allocate one library. It's basically the same as what we had before we added support for the TST.JCL library.

This code is essentially the same for all source libraries. It could be factored out into a PROC.

```
 File  Edit  Edit_Settings  Menu  Utilities  Compilers  Test  Help
--------------------------------------------------------------------------------
EDIT       IBMUSER.LAB.JCL(CRSRCLIB) - 01.03               Columns 00001 00072
Command ===> _____________________________________________ Scroll ===> CSR
****** ***************************** Top of Data ******************************
000001 //IBMUSERL JOB ,'ALLOC SOURCE LIBRARY',
000002 //           CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
000003 //           NOTIFY=&SYSUID
000004 //CLEANUP  EXEC PGM=IEFBR14
000005 //DD1      DD DSN=&SYSUID..DEV.JCL,
000006 //           DISP=(MOD,DELETE,DELETE),
000007 //           UNIT=SYSDA,VOL=SER=USRVS1,
000008 //           SPACE=(TRK,(1,0)),
000009 //           DCB=&SYSUID..MODELDCB.DCBLIB80,
000010 //           DSNTYPE=LIBRARY
000011 //ALLOC    EXEC PGM=IEFBR14
000012 //DD1      DD DSN=&SYSUID..DEV.JCL,
000013 //           DISP=(NEW,CATLG,DELETE),
000014 //           UNIT=SYSDA,VOL=SER=USRVS1,
000015 //           SPACE=(TRK,(1,1),RLSE),
000016 //           DCB=&SYSUID..MODELDCB.DCBLIB80,
000017 //           DSNTYPE=LIBRARY
****** ***************************** Bottom of Data ***************************
```

REFACTORING THE JCL

Next, we'll define these two steps, CLEANUP and ALLOC, as an inline PROC and execute it for the DEV.JCL library.

Line 4 now contains a PROC statement with the name MAKELIB.

Line 19 denotes the end of the PROC with the PEND statement.

Then we added a job step named DEVJCL that executes PROC MAKELIB.

```
 File  Edit  Edit_Settings  Menu  Utilities  Compilers  Test  Help
--------------------------------------------------------------------------------
EDIT       IBMUSER.LAB.JCL(CRSRCLIB) - 01.03               Columns 00001 00072
Command ===>                                                 Scroll ===> CSR
****** ***************************** Top of Data ******************************
000001 //IBMUSERL JOB ,'ALLOC SOURCE LIBRARY',
000002 //            CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
000003 //            NOTIFY=&SYSUID
000004 //MAKELIB  PROC
000005 //CLEANUP  EXEC PGM=IEFBR14
000006 //DD1      DD DSN=&SYSUID..DEV.JCL,
000007 //            DISP=(MOD,DELETE,DELETE),
000008 //            UNIT=SYSDA,VOL=SER=USRVS1,
000009 //            SPACE=(TRK,(1,0)),
000010 //            DCB=&SYSUID..MODELDCB.DCBLIB80,
000011 //            DSNTYPE=LIBRARY
000012 //ALLOC    EXEC PGM=IEFBR14
000013 //DD1      DD DSN=&SYSUID..DEV.JCL,
000014 //            DISP=(NEW,CATLG,DELETE),
000015 //            UNIT=SYSDA,VOL=SER=USRVS1,
000016 //            SPACE=(TRK,(1,1),RLSE),
000017 //            DCB=&SYSUID..MODELDCB.DCBLIB80,
000018 //            DSNTYPE=LIBRARY
000019 //         PEND
000020 //DEVJCL   EXEC MAKELIB
****** **************************** Bottom of Data ****************************
```

REFACTORING THE JCL

That's fine for allocating DEV.JCL, but what about TST.JCL?

We can use JCL symbols to set different values for the library's data set name.

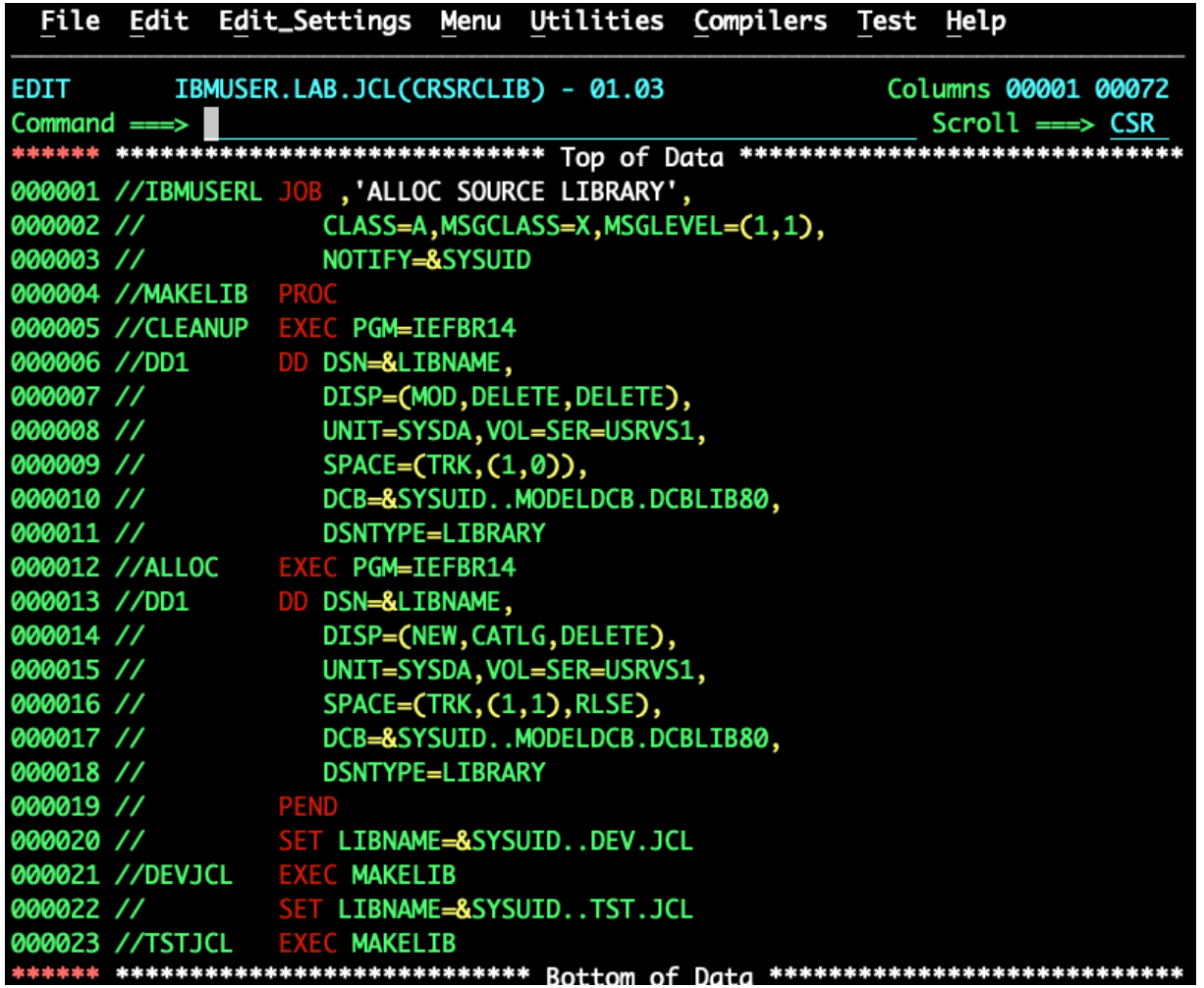
You've seen how the system-managed symbol SYSUID can be used in JCL. We can also define our own symbols.

We changed the DSN values on lines 6 and 13 to &LIBNAME. That's a name we made up.

Before calling the PROC, we set the value of symbol LIBNAME to the data set name we want to allocate.

In job step DEVJCL, LIBNAME is set to &SYSUID..DEV.JCL.

In job step TSTJCL, LIBNAME is set to &SYSUID..TST.JCL.



REFACTORING THE JCL

The other way to set symbol values is by passing them on the EXEC statement.

Lines 20 and 21 of this example illustrate this method.

```
 File  Edit  Edit_Settings  Menu  Utilities  Compilers  Test  Help
EDIT       IBMUSER.LAB.JCL(CRSRCLIB) - 01.04            Columns 00001 00072
Command ===>                                             Scroll ===> CSR
****** ***************************** Top of Data ******************************
000001 //IBMUSERL JOB ,'ALLOC SOURCE LIBRARY',
000002 //            CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
000003 //            NOTIFY=&SYSUID
000004 //MAKELIB  PROC
000005 //CLEANUP  EXEC PGM=IEFBR14
000006 //DD1      DD DSN=&LIBNAME,
000007 //            DISP=(MOD,DELETE,DELETE),
000008 //            UNIT=SYSDA,VOL=SER=USRVS1,
000009 //            SPACE=(TRK,(1,0)),
000010 //            DCB=&SYSUID..MODELDCB.DCBLIB80,
000011 //            DSNTYPE=LIBRARY
000012 //ALLOC    EXEC PGM=IEFBR14
000013 //DD1      DD DSN=&LIBNAME,
000014 //            DISP=(NEW,CATLG,DELETE),
000015 //            UNIT=SYSDA,VOL=SER=USRVS1,
000016 //            SPACE=(TRK,(1,1),RLSE),
000017 //            DCB=&SYSUID..MODELDCB.DCBLIB80,
000018 //            DSNTYPE=LIBRARY
000019 //         PEND
000020 //DEVJCL   EXEC MAKELIB,LIBNAME=&SYSUID..DEV.JCL
000021 //TSTJCL   EXEC MAKELIB,LIBNAME=&SYSUID..TST.JCL
****** ***************************** Bottom of Data ***************************
```

REFACTORING THE JCL

A little bit of cleanup. Added a comment explaining the purpose of the job. Changed DDNAMEs on lines 9 and 16 to values that are hopefully a little more descriptive than "DD1".

```
 File  Edit  Edit_Settings  Menu  Utilities  Compilers  Test  Help
------------------------------------------------------------------------------
EDIT       IBMUSER.LAB.JCL(CRSRCLIB) - 01.04            Columns 00001 00072
Command ===> _____________________________________________ Scroll ===> CSR
****** ***************************** Top of Data ******************************
000001 //IBMUSERL JOB ,'ALLOC SOURCE LIBRARIES',
000002 //           CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
000003 //           NOTIFY=&SYSUID
000004 //*********************************************************************
000005 //* ALLOCATE SOURCE LIBRARIES FOR AN APPLICTION DEVELOPER.
000006 //*********************************************************************
000007 //MAKELIB  PROC
000008 //CLEANUP  EXEC PGM=IEFBR14
000009 //OLDLIB   DD DSN=&LIBNAME,
000010 //            DISP=(MOD,DELETE,DELETE),
000011 //            UNIT=SYSDA,VOL=SER=USRVS1,
000012 //            SPACE=(TRK,(1,0)),
000013 //            DCB=&SYSUID..MODELDCB.DCBLIB80,
000014 //            DSNTYPE=LIBRARY
000015 //ALLOC    EXEC PGM=IEFBR14
000016 //NEWLIB   DD DSN=&LIBNAME,
000017 //            DISP=(NEW,CATLG,DELETE),
000018 //            UNIT=SYSDA,VOL=SER=USRVS1,
000019 //            SPACE=(TRK,(1,1),RLSE),
000020 //            DCB=&SYSUID..MODELDCB.DCBLIB80,
000021 //            DSNTYPE=LIBRARY
000022 //         PEND
000023 //DEVJCL   EXEC MAKELIB,LIBNAME=&SYSUID..DEV.JCL
000024 //TSTJCL   EXEC MAKELIB,LIBNAME=&SYSUID..TST.JCL
****** ***************************** Bottom of Data ***************************
```

REFACTORING THE JCL

It's likely that all these source libraries don't need to be the same size. Let's enhance the PROC so that it takes parameters for the primary and secondary space allocation values.

On the PROC statement we add two symbol names, PRI and SEC, and give them default values.

On line 23 we invoke the PROC and override the values of PRI and SEC.

On line 24 we invoke the PROC without mentioning symbols PRI and SEC. JES uses the default values coded in the PROC.

```
 File  Edit  Edit_Settings  Menu  Utilities  Compilers  Test  Help
--------------------------------------------------------------------------------
EDIT       IBMUSER.LAB.JCL(CRSRCLIB) - 01.04               Columns 00001 00072
Command ===>                                                 Scroll ===> CSR
****** ***************************** Top of Data ******************************
000001 //IBMUSERL JOB ,'ALLOC SOURCE LIBS',
000002 //           CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
000003 //           NOTIFY=&SYSUID
000004 //*********************************************************************
000005 //* ALLOCATE SOURCE LIBRARIES FOR AN APPLICTION DEVELOPER.
000006 //*********************************************************************
000007 //MAKELIB  PROC PRI=1,SEC=1
000008 //CLEANUP  EXEC PGM=IEFBR14
000009 //OLDLIB   DD DSN=&LIBNAME,
000010 //            DISP=(MOD,DELETE,DELETE),
000011 //            UNIT=SYSDA,VOL=SER=USRVS1,
000012 //            SPACE=(TRK,(1,0)),
000013 //            DCB=&SYSUID..MODELDCB.DCBLIB80,
000014 //            DSNTYPE=LIBRARY
000015 //ALLOC    EXEC PGM=IEFBR14
000016 //NEWLIB   DD DSN=&LIBNAME,
000017 //            DISP=(NEW,CATLG,DELETE),
000018 //            UNIT=SYSDA,VOL=SER=USRVS1,
000019 //            SPACE=(TRK,(&PRI,&SEC),RLSE),
000020 //            DCB=&SYSUID..MODELDCB.DCBLIB80,
000021 //            DSNTYPE=LIBRARY
000022 //         PEND
000023 //DEVJCL   EXEC MAKELIB,LIBNAME=&SYSUID..DEV.JCL,PRI=2,SEC=2
000024 //TSTJCL   EXEC MAKELIB,LIBNAME=&SYSUID..TST.JCL
****** ***************************** Bottom of Data ***************************
```

REFACTORING THE JCL

On the right is our catalogued procedure, MAKELIB, in our PROCLIB.

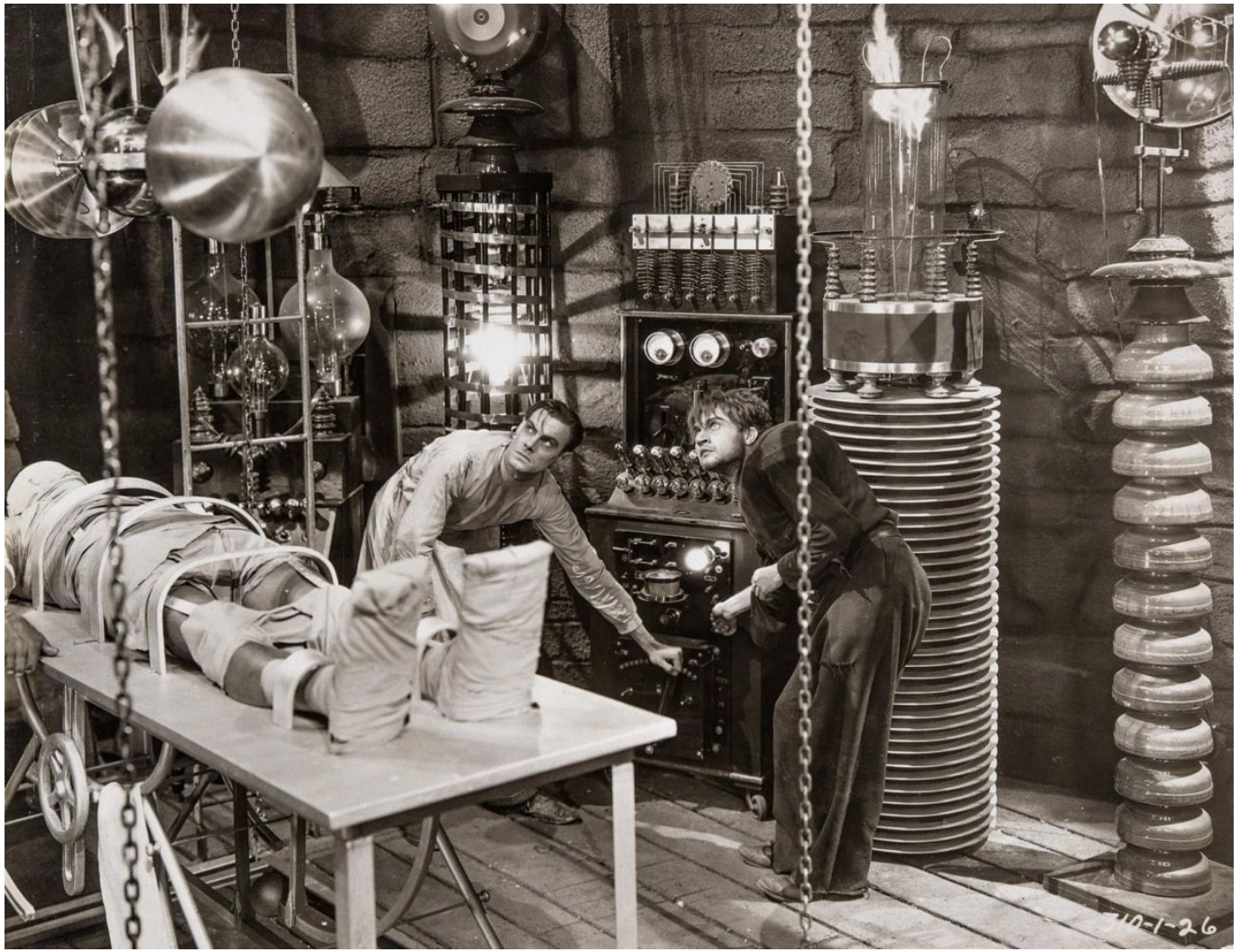
The JCL to call it is below.

We need to add a special JCL statement, JCLLIB, to tell JES to search our PROCLIB ahead of the system default PROCLIB. It will still search the default PROCLIB if it doesn't find a PROC we reference in our PROCLIB.

```
 File  Edit  Edit_Settings  Menu  Utilities  Compilers  Test  Help
EDIT       IBMUSER.LAB.PROCLIB(MAKELIB) - 01.00            Columns 00001 00072
Command ===>                                                Scroll ===> CSR
****** ***************************** Top of Data ******************************
000001 //*********************************************************************
000002 //* ALLOCATE SOURCE LIBRARIES FOR AN APPLICTION DEVELOPER.
000003 //*********************************************************************
000004 //MAKELIB  PROC PRI=1,SEC=1
000005 //CLEANUP  EXEC PGM=IEFBR14
000006 //OLDLIB   DD DSN=&LIBNAME,
000007 //            DISP=(MOD,DELETE,DELETE),
000008 //            UNIT=SYSDA,VOL=SER=USRVS1,
000009 //            SPACE=(TRK,(1,0)),
000010 //            DCB=&SYSUID..MODELDCB.DCBLIB80,
000011 //            DSNTYPE=LIBRARY
000012 //ALLOC    EXEC PGM=IEFBR14
         DD DSN=&LIBNAME,
            DISP=(NEW,CATLG,DELETE),
            UNIT=SYSDA,VOL=SER=USRVS1,
            SPACE=(TRK,(&PRI,&SEC),RLSE),
            DCB=&SYSUID..MODELDCB.DCBLIB80,
            DSNTYPE=LIBRARY
         PEND
****************************** Bottom of Data ******************************
```

```
 File  Edit  Edit_Settings  Menu  Utilities  Compilers  Test  Help
EDIT       IBMUSER.LAB.JCL(CRSRCLIB) - 01.10               Columns 00001 00072
Command ===>                                                Scroll ===> CSR
****** ***************************** Top of Data ******************************
000001 //IBMUSERL JOB ,'ALLOC SOURCE LIBS',
000002 //            CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
000003 //            NOTIFY=&SYSUID
000004 //         JCLLIB ORDER=(&SYSUID..LAB.PROCLIB)
000005 //*********************************************************************
000006 //* ALLOCATE SOURCE LIBRARIES FOR AN APPLICTION DEVELOPER.
000007 //*********************************************************************
000008 //DEVJCL   EXEC MAKELIB,LIBNAME=&SYSUID..DEV.JCL,PRI=2,SEC=2
000009 //TSTJCL   EXEC MAKELIB,LIBNAME=&SYSUID..TST.JCL
****** ***************************** Bottom of Data ***************************
```

LAB Z/OS 14: REFACTOR YOUR PROVISIONING JOB



CREATING DEVELOPER PROGRAM LIBRARIES

The members of a *program library* are executables, known as Program Objects. A program library is a PDSE, or DSNTYPE=LIBRARY.

Executables in the appropriate format for a program library are produced by the *binder*, which processes the output from assemblers and compilers to link static subprograms and otherwise prepare the object code for execution.

You also need to be aware of an earlier format, still supported for backward compatibility, called a *load library* or LOADLIB. A load library is a PDS.

Object programs in the appropriate format for a load library are produced by the *linkage editor*, which processes the output from assemblers and compilers similar to the way the binder does.

Oldtimers, as well as many system messages and labels you will find here and there, refer to both types of libraries as *load libraries* and both types of objects as *load modules*.

Members of a load library are not immediately executable. z/OS must use metadata stored in the library at load time to make the object code ready to run. There are other limitations of the format that make it less desirable than a program library, such as a limit on the number of directory entries it may contain.

For anything new (for some definition of "new") we want to use program libraries, and only use load libraries for older systems that require it.

CREATING DEVELOPER PROGRAM LIBRARIES

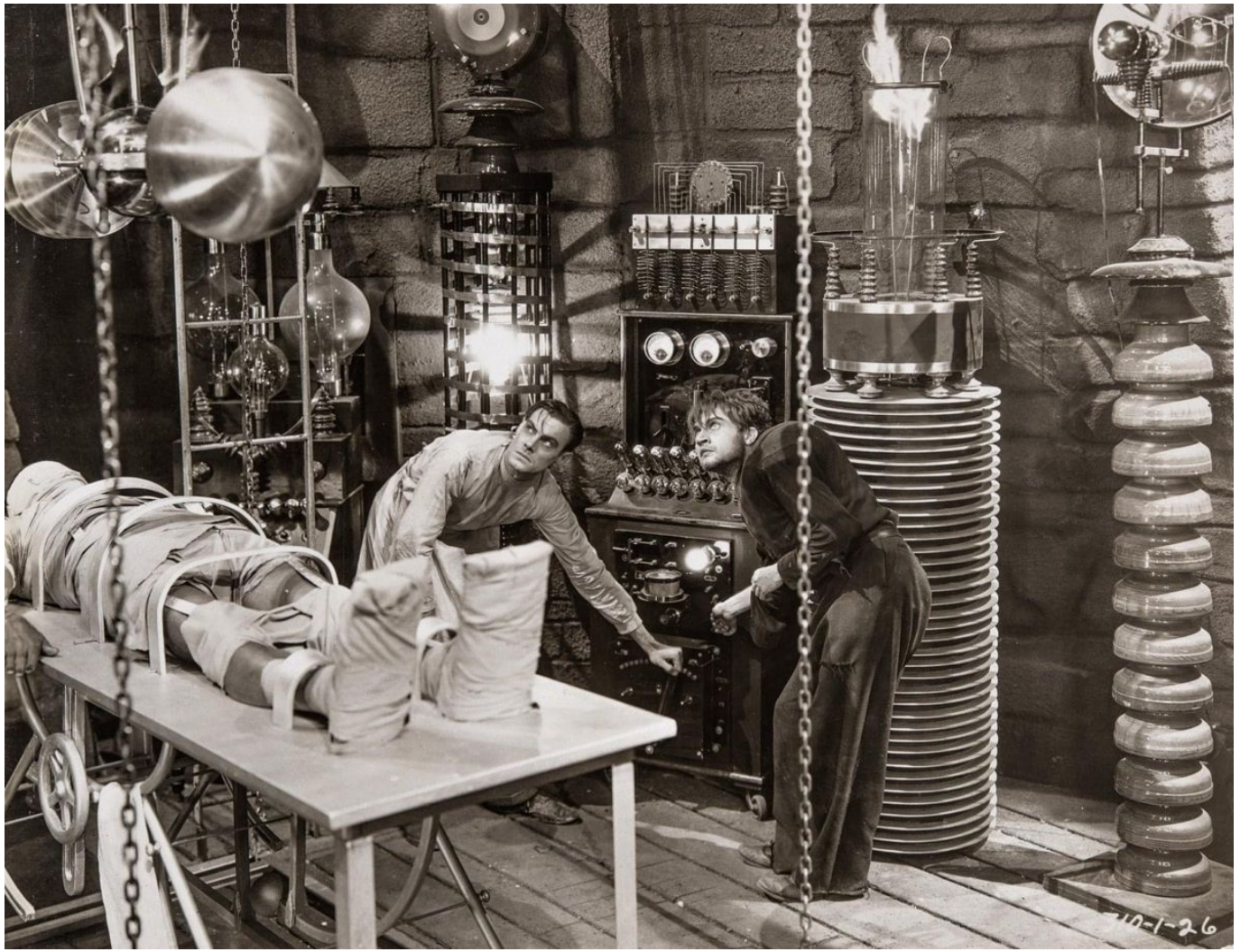
For our lab work we are making the assumption that our developers will need program libraries for their work in progress and for programs ready for testing and integration. This may be different in your actual work environment.

In our lab environment we decided to name these libraries **`<userid>.DEV.PROGLIB`** and **`<userid>.TST.PROGLIB`**.

The data set attributes for a program library are:

Data Set Organization	PO	Partitioned Organization
Record Format	U	Undefined
Logical Record Length	0	(byte stream)
Block Size	4096	(memory page size)
Data Set Name Type	LIBRARY	PDSE

LAB Z/OS 15: WRITE JCL TO CREATE PROGRAM LIBS



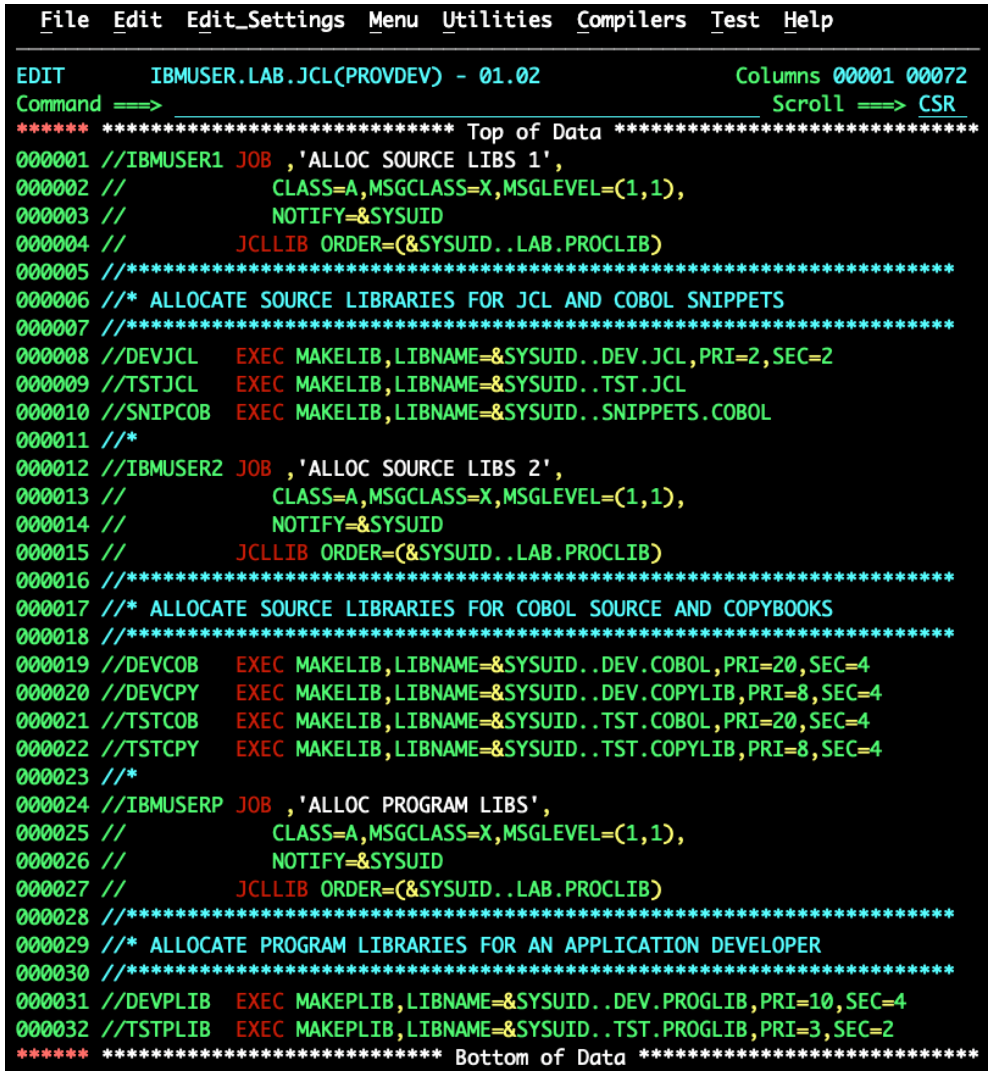
A SINGLE PROVISIONING JOB

You'll hear the term *job stream* quite a lot. The reason for it is a single JCL file can contain more than one job. JES will start each job as a separate process. It's a "stream of jobs."

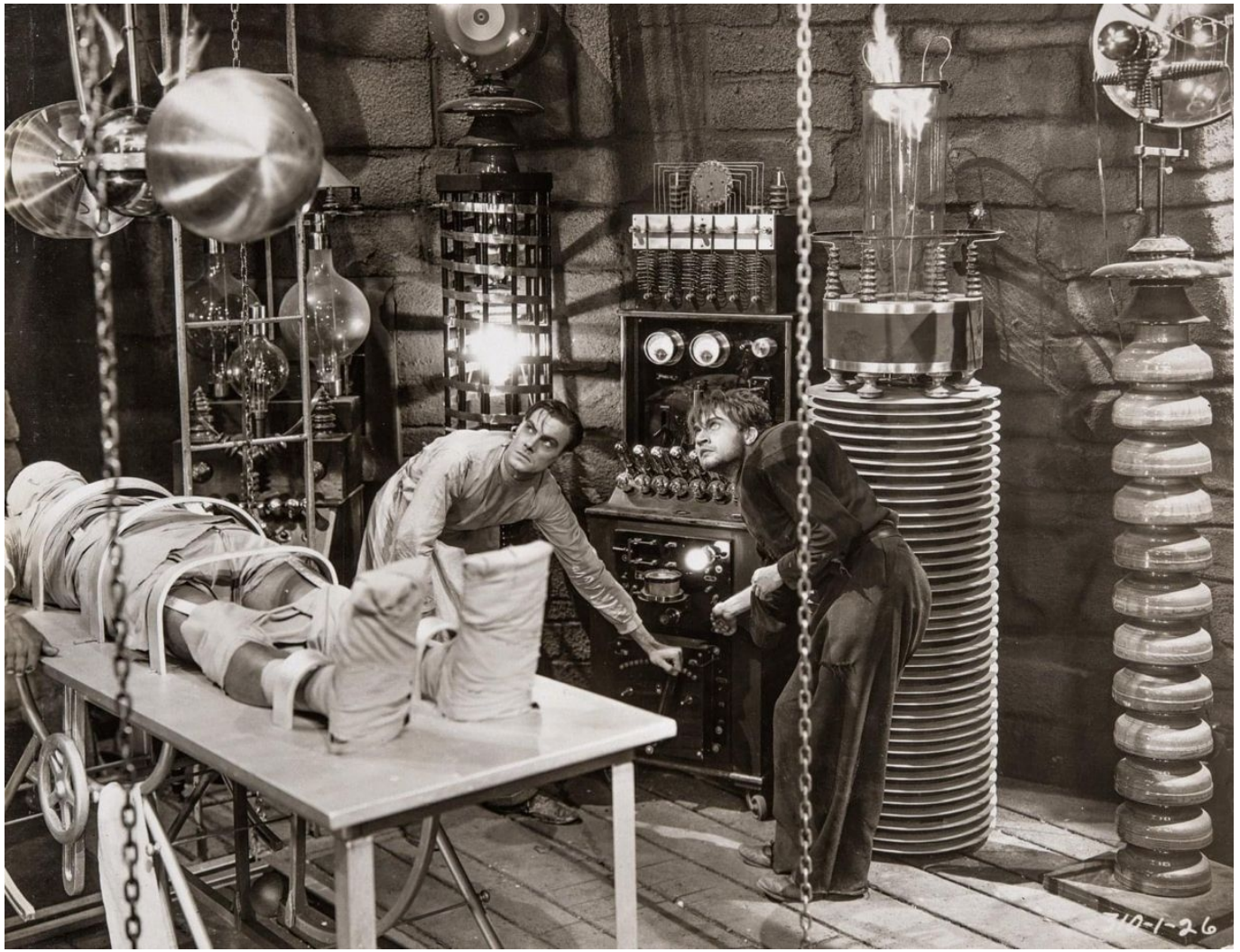
In our case, the operations carried out by our library provisioning jobs don't interfere with each other and don't depend on each other in any way. It's safe to run them all at the same time.

The example at right doesn't include all the libraries in our hypothetical developer setup, but it illustrates how you can stack multiple jobs in the same JCL file.

The example contains three jobs. The first two allocate different source libraries, creating three or four in each job. The last job in the stream creates the program libraries.



LAB Z/OS 16: SET UP A ONE-SHOT PROVISIONING JOB



WHAT YOU KNOW AND WHAT YOU CAN DO

- You can write an idempotent job that creates a PDSE Library.
- You can define JCL Catalogued Procedures (PROCS) to enable re-use of job steps.
- You can define and use JCL symbols.
- You can assemble multiple jobs into a job stream.
- You can locate and interpret spooler output from batch jobs using SDSF.
- You can use several ISPF Editor commands to create and manipulate JCL.

WHAT'S NEXT

- Using IBM utilities to seed developer libraries with sample members.